



Proudly supported by



edunet
foundation



Module - 1

Introduction to AI and ML Part 2



DISCLAIMER

The content is curated from online/offline resources and used for educational purpose only.

Learning Outcome

- Explain the role of unsupervised learning in sustainability by highlighting its potential in analyzing large, unlabeled environmental datasets.
- Use clustering techniques to identify patterns in sustainability data, such as categorizing regions based on energy consumption, air quality, or waste production.
- Apply evaluation metrics and visualization techniques to interpret the results of unsupervised models in the context of sustainability goals.
- Use TensorFlow/PyTorch to build a simple ML model.



Source



What is Unsupervised Learning

- Unsupervised learning is a type of machine learning where algorithms analyze and model data without any predefined labels or outputs.
- Unlike supervised learning, which uses labelled data to train models, unsupervised learning deals with unlabelled data, exploring patterns, structures, and relationships within the data on its own.
- Unsupervised learning techniques include clustering, dimensionality reduction, and anomaly detection.

Unlabeled data

Labelled data



Labelled data



Unlabelled data



Unlabeled data refers to a dataset where the target or outcome variable (the "label") is not provided. In other words, for each data point in the dataset, there is no predefined "correct" answer or class associated with it. Unlabeled data consists only of input features (such as text, images, or numerical values) without any associated labels, categories, or outcomes.

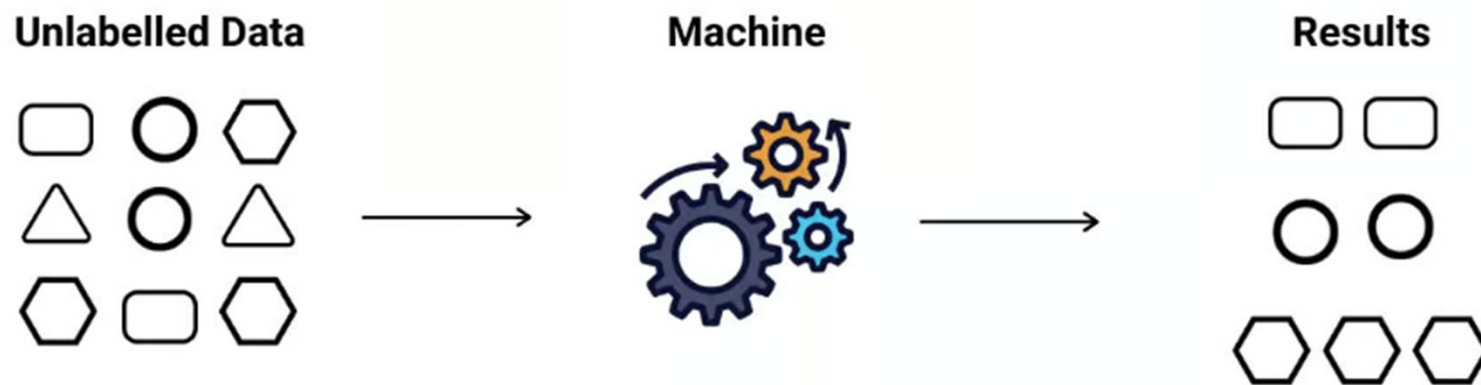
Customer Id	Age	Edu	Years Employed	Income	Card Debt	Other Debt	Address	DebtIncomeRatio
1	41	2	6	19	0.124	1.073	NBA001	6.3
2	47	1	26	100	4.582	8.218	NBA021	12.8
3	33	2	10	57	6.111	5.802	NBA013	20.9
4	29	2	4	19	0.681	0.516	NBA009	6.3
5	47	1	31	253	9.308	8.908	NBA008	7.2
6	40	1	23	81	0.998	7.831	NBA016	10.9
7	38	2	4	56	0.442	0.454	NBA013	1.6
8	42	3	0	64	0.279	3.945	NBA009	6.6
9	26	1	5	18	0.575	2.215	NBA006	15.5
10	47	3	23	115	0.653	3.947	NBA011	4
11	44	3	8	88	0.285	5.083	NBA010	6.1
12	34	2	9	40	0.374	0.266	NBA003	1.6

unlabeled

Unlabelled data is crucial in advancing the sustainability of green technology, as it provides vast, untapped resources for identifying environmental patterns, improving resource management, and driving innovative eco-friendly solutions.



How Unsupervised learning works?



- Data Collection and Preparation
- Selecting an Unsupervised Learning Algorithm
- Pattern Identification
- Evaluation and Interpretation



Types of Unsupervised Learning and Algorithms

Clustering

Clustering is a fundamental technique in unsupervised learning where the algorithm groups data points into clusters based on their similarity. Each cluster ideally represents a group of items with common characteristics, making it particularly useful in areas of green technology where there is a need to categorize vast data efficiently.

Dimensionality Reduction

Dimensionality reduction simplifies high-dimensional datasets by reducing the number of variables while preserving important information. This technique is essential for green technology applications, where datasets can be large and complex—like satellite images, climate datasets, or energy grids.



Unsupervised Learning Algorithms

- K-Means
- Hierarchical Clustering
- Principal Component Analysis (PCA)
- t-SNE (t-Distributed Stochastic Neighbor Embedding)



└ Evaluation Techniques

- Silhouette Score
- Adjusted Rand Index (ARI)
- Normalized Mutual Information (NMI)



Evaluation Techniques

Silhouette Score

The formula for the Silhouette Score $S(i)$ of a single data point i is,
$$S(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where:

- $a(i)$ is the average distance between point i and all other points in the same cluster (intra-cluster distance).
- $b(i)$ is the minimum average distance from point i to all points in the nearest neighbouring cluster (inter-cluster distance).

The Silhouette Score ranges from -1 to 1:

- **1**: indicates that the data points are well-clustered and far from neighbouring clusters.
- **0**: suggests that clusters are overlapping or indistinguishable.
- **-1**: indicates that the clustering is poor, as data points may be assigned to the wrong cluster



Evaluation Techniques

Adjusted Rand Index (ARI)

The ARI formula is,
$$ARI = \frac{RI - \mathbb{E}[RI]}{\max(RI) - \mathbb{E}[RI]}$$

where:

- **RI (Rand Index)** counts the agreement between pairs of elements, considering pairs that are either in the same cluster in both clustering or in different clusters in both.
- $\mathbb{E}[RI]$ represents the expected RI under a random model.

The ARI ranges from -1 to 1:

- **1:** indicates perfect clustering agreement with ground truth.
- **0:** reflects random labelling.
- **Negative values** indicate worse-than-random labelling.



Evaluation Techniques

Normalized Mutual Information (NMI)

The NMI formula is,
$$\text{NMI}(U, V) = \frac{2 \cdot I(U, V)}{H(U) + H(V)}$$

where:

- U and V are two clustering assignments.
- $I(U, V)$ is the mutual information between U and V, measuring the amount of shared information.
- $H(U)$ and $H(V)$ are the entropies of U and V, representing the uncertainty in each clustering assignment.

NMI ranges from 0 to 1:

- **1**: indicates a perfect match between the two clustering.
- **0**: suggests that the clustering are independent.



Clustering

What is Clustering?

Clustering involves partitioning a dataset into groups or clusters, where each cluster contains data points that are more similar to each other than to those in other clusters. Similarity is typically measured based on distance metrics, such as Euclidean or Manhattan distance, depending on the data and objective.

Clustering Algorithms

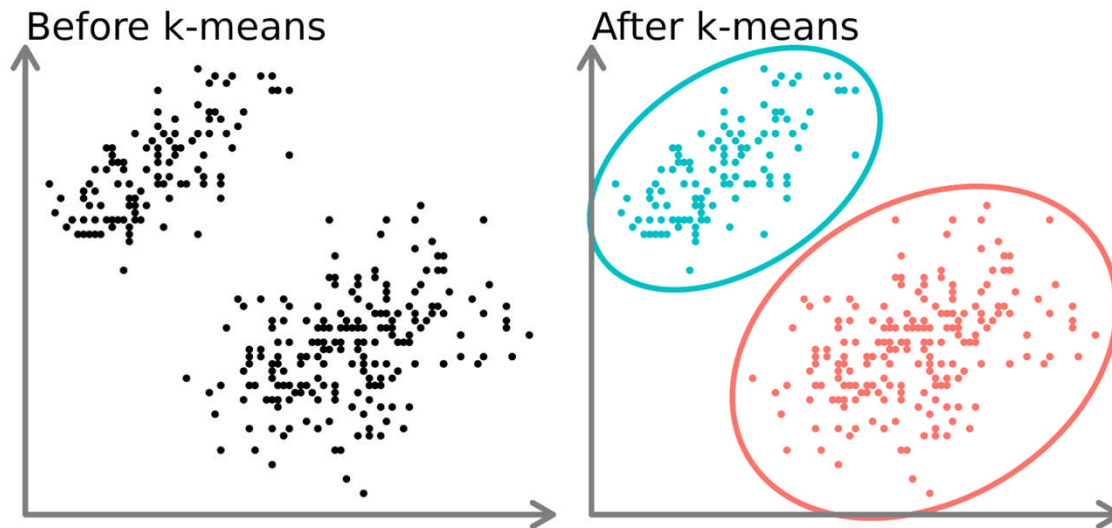
Common clustering algorithms include,

- K-Means Clustering
- Hierarchical Clustering



Understanding and Implementation k-means clustering

Understanding of k-means clustering



Clustering models aim to group data into distinct “clusters” or groups. This can both serve as an interesting view in an analysis or can serve as a feature in a supervised learning algorithm.



Understanding and Implementation k-means clustering

Understanding of k-means clustering

The main goal of K-means clustering is to minimize the sum of squared distances between each data point and the centroid of its assigned cluster.

This is the cost function,

$$J = \sum_{i=1}^n \sum_{k=1}^K \mathbb{I}(c_i = k) \|x_i - \mu_k\|^2$$

Where:

- n = number of data points.
- K = number of clusters.
- c_i = cluster label assigned to data point i .
- x_i = feature vector of data point i .
- μ_k = centroid of cluster k .
- $\mathbb{I}(c_i = k)$ = indicator function that is 1 if the point i is assigned to cluster k and 0 otherwise.
- $\|x_i - \mu_k\|$ = Euclidean distance between data point i and centroid μ_k .

Euclidean Distance Formula

The distance metric used in K-means is typically the Euclidean distance

$$\|x_i - \mu_k\| = \sqrt{\sum_{j=1}^d (x_{ij} - \mu_{kj})^2}$$

Where:

- x_{ij} is the j -th feature of data point x_i .
- μ_{kj} is the j -th feature of centroid μ_k .
- d is the number of features.



Understanding and Implementation k-means clustering

Implementation of k-means clustering

- Import Necessary Libraries
- Load dataset for Environmental Factors
- Preprocess the Data
- Apply K-Means Clustering
- Elbow Method for Choosing k
- Fitting K-Means with Optimal k
- Evaluating the Clustering Performance
- Visualize Clusters

Classroom Hands-on

7. k-means clustering



Understanding and Implementation hierarchical clustering

Understanding of hierarchical clustering

Hierarchical clustering is a powerful unsupervised machine learning technique used to group data into clusters based on their similarity.

Hierarchical clustering builds a hierarchy of clusters by either a bottom-up (agglomerative) or top-down (divisive) approach:

- Agglomerative: Starts with each data point as its own cluster and progressively merges the most similar clusters until all data points belong to one cluster.
- Divisive: Starts with all data points in a single cluster and recursively splits the data into smaller clusters.



Understanding and Implementation hierarchical clustering

Understanding of hierarchical clustering

Overview of how hierarchical clustering works and the formulas involved,

Distance Calculation

Euclidean Distance Formula,

For two points $\mathbf{x} = (x_1, x_2, \dots, x_n)$ and $\mathbf{y} = (y_1, y_2, \dots, y_n)$ in an n -dimensional space

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

For other types of data, you might use different distance measures such as Manhattan distance, Cosine similarity, or Jaccard similarity depending on the nature of the data.



Understanding and Implementation hierarchical clustering

Understanding of hierarchical clustering

Agglomerative (Bottom-Up) Clustering Process

In agglomerative hierarchical clustering, the steps to form clusters are as follows:

- Start with individual points as clusters: Each data point is considered a separate cluster.
- Calculate the distance between all pairs of clusters using a distance metric (e.g., Euclidean).
- Merge the two closest clusters: At each step, the two clusters with the smallest distance between them are merged.
- Repeat steps 2 and 3 until all points are in one cluster.



Understanding and Implementation hierarchical clustering

Understanding of hierarchical clustering

Divisive (Top-Down) Clustering Process

In divisive hierarchical clustering, the process is the reverse of agglomerative clustering:

- Start with all points in one cluster.
- Split the cluster into two sub-clusters based on some splitting criterion (e.g., maximizing the between-cluster variance).
- Repeat the process: Split the clusters iteratively until each data point is in its own cluster.



Understanding and Implementation hierarchical clustering

Implementation of hierarchical clustering

To implement Hierarchical Clustering on a dataset containing both Environmental Impact Factors (like CO₂ emissions and waste) and Socioeconomic Indicators (like GDP and population).

- Load Dataset
- Preprocess the Data
- Hierarchical Clustering
- Fit the Model and Create Dendrogram
- Apply Agglomerative Clustering
- Evaluate the Clustering
- Visualizing the Clusters

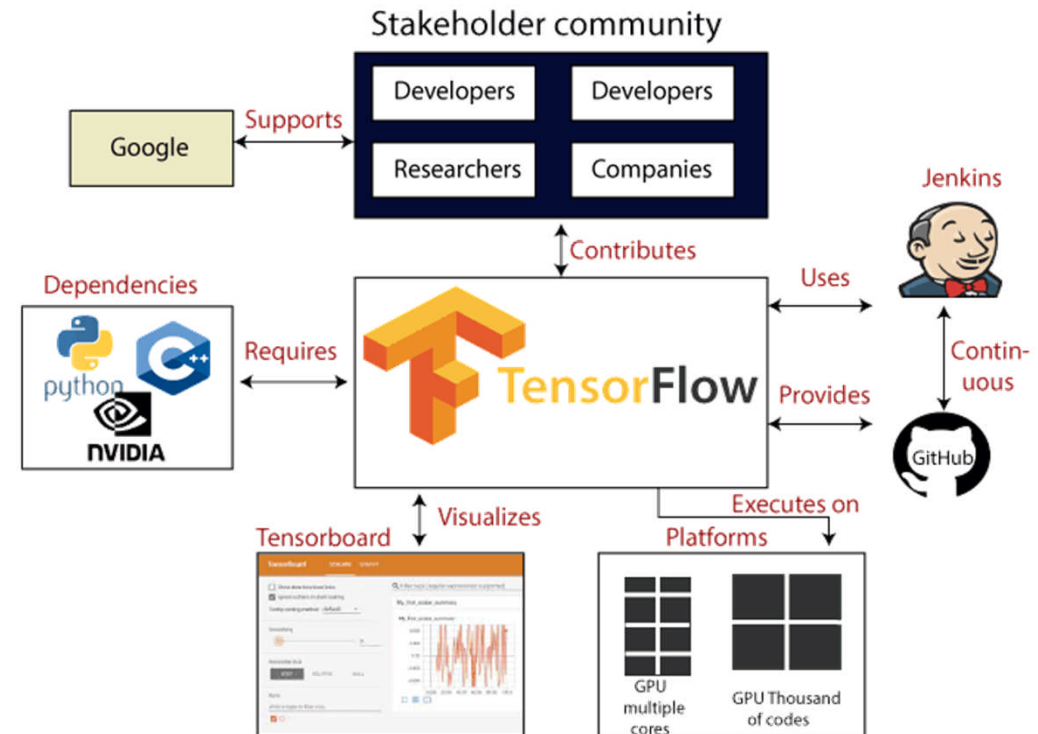


Dimensionality Reduction

- Dimensionality reduction is a process in unsupervised learning that aims to reduce the number of features (or dimensions) in a dataset while preserving as much relevant information as possible.
- This is particularly useful when dealing with high-dimensional data, where the number of features is large.
- Dimensionality reduction techniques help simplify the data, make it easier to visualize, and often improve the efficiency of machine learning algorithms.

What is TensorFlow?

- Open-source library developed by Google
- Designed for deep learning and supports traditional ML
- Works with multi-dimensional data structures called tensors
- Uses data flow graphs with nodes and edges
- Enables distributed computing across CPUs/GPUs



Source

How TensorFlow Works

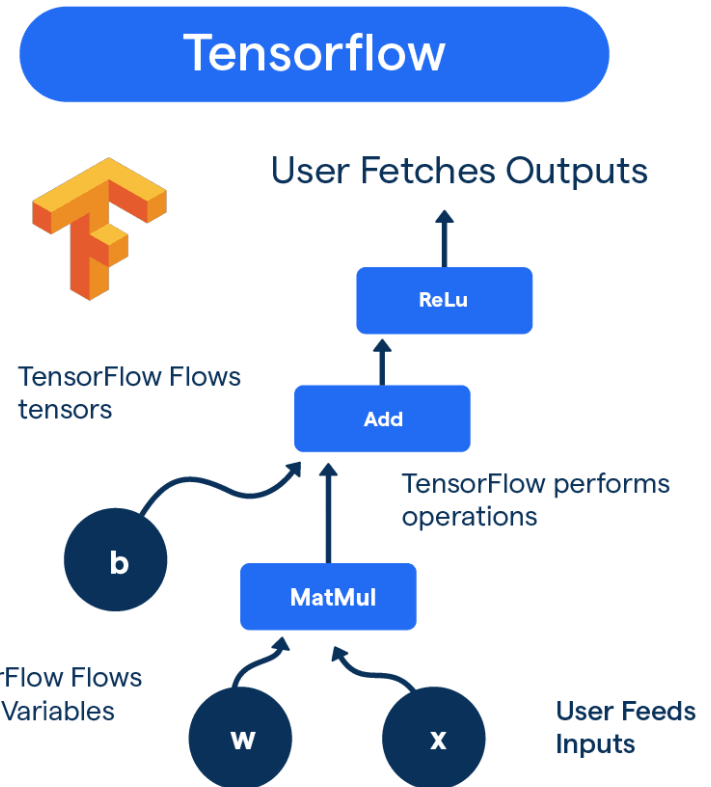
- Key components are Data Preprocessing, Model Construction, Model Training

Operates using:

- Computational Graphs (dataflow-based)
- Eager Execution (runs operations immediately)
- Uses tensors (multi-dimensional arrays) for computation
- Supports execution on CPUs, GPUs, desktops, mobile & cloud

Tools:

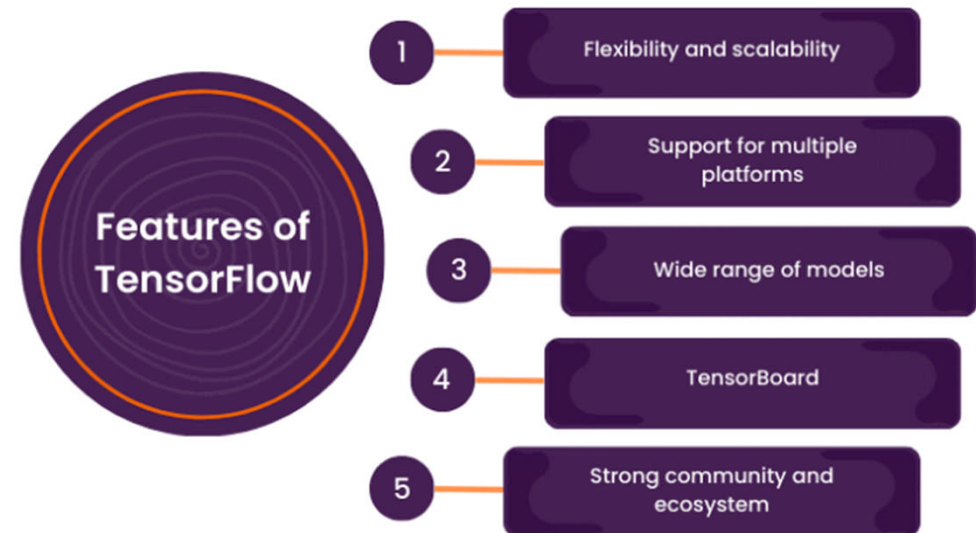
- **TensorBoard** – visualizes training process and performance
- **Keras API** – simplifies model building and experimentation



Source

Why Use TensorFlow?

- Flexible & Powerful platform for ML model building and deployment
- Simplified development with Keras API
- Advanced features: Eager Execution & Distributed Training
- Seamless deployment on Cloud, Mobile, Edge, Web
- Supports production pipelines with TFX, mobile optimization with TensorFlow Lite, and web deployment with TensorFlow.js
- Rich tools for research: Functional API, Model Subclassing, and Pre-trained Models

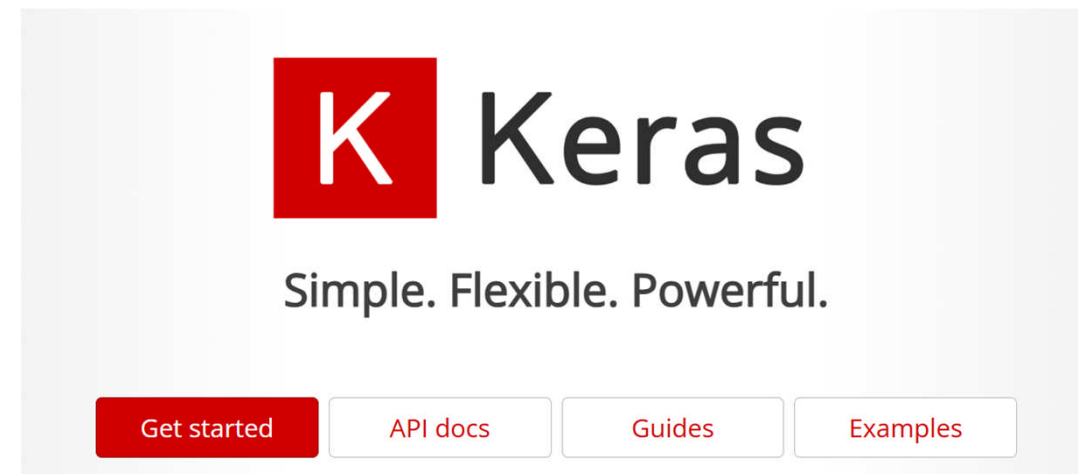


Source



Keras as a High-Level API

- High-level API of TensorFlow for easy deep learning model development
- **Simple & Modular:** Ideal for rapid experimentation
- **Cross-platform support:** Compatible with CPUs, GPUs, TPUs, and mobile
- **Rich library of predefined components:** Layers, optimizers, loss functions
- **Multiple model-building options:** Sequential, Functional API, Subclassing
- **Built-in training methods:** fit(), evaluate(), predict()
- Efficient data preprocessing tools for images, text, and numbers



Source

<https://keras.io/>



Key Features of Keras

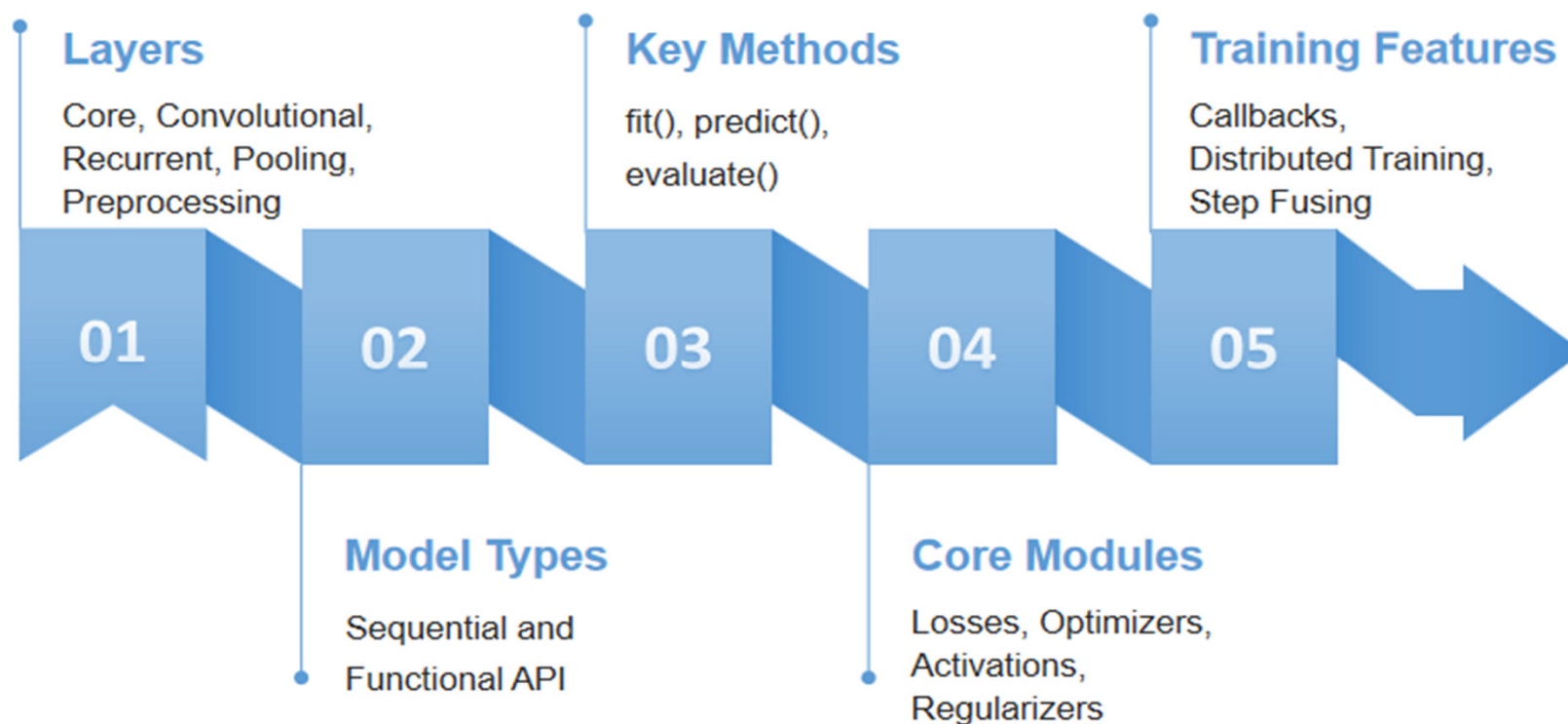
- **Cross-Platform Scalability:** Runs on various hardware, including CPUs, GPUs, TPUs, and mobile devices.
- **Predefined Components:** Offers a large set of built-in layers, loss functions, optimizers, and activation functions.
- **Multiple Model APIs:** Supports Sequential API for simple layer stacking, Functional API for complex architectures, and Model Subclassing for full customization.
- **Built-in Training Methods:** Includes `fit()`, `evaluate()`, and `predict()` for efficient model training and evaluation.
- **Data Preprocessing:** Provides tools for handling images, text, and numerical data efficiently.

Source

https://www.researchgate.net/figure/Illustrative-image-of-artificial-neural-networks-with-their-working-mechanism-as-like_fig13_380136894



Keras API Components



TensorFlow Installation & Setup

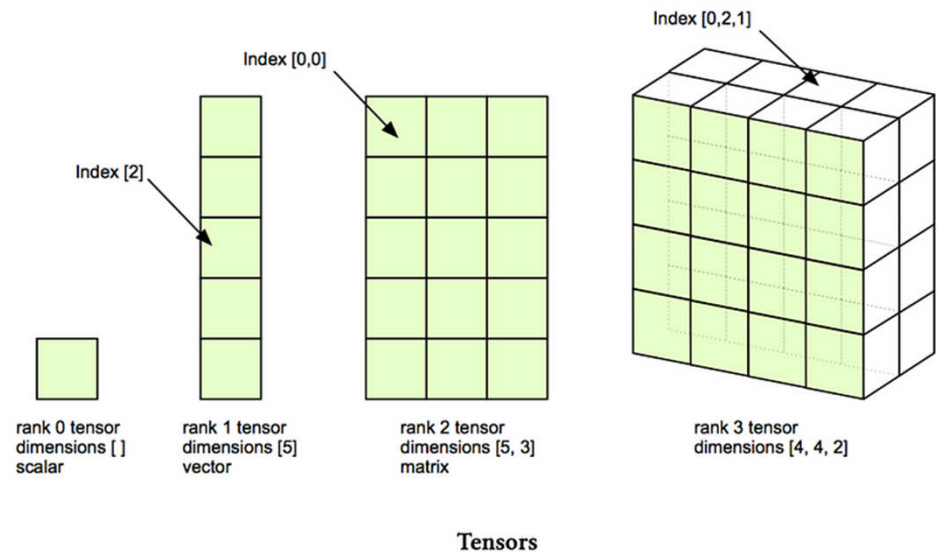
- Check if pip is installed (pip --version)
- (Optional) Create virtual environment (python -m venv tensorflow_env)
- Activate environment (activate on Windows / source on macOS/Linux)
- Upgrade pip (pip install --upgrade pip)
- Install TensorFlow (pip install tensorflow)
- Verify installation (import tensorflow as tf, print(tf.__version__))

```
Downloading werkzeug-3.1.3-py3-none-any.whl (224 kB)
 224.5/224.5 kB 3.4 MB/s eta 0:00:00
Downloading wheel-0.45.1-py3-none-any.whl (72 kB)
 72.5/72.5 kB 3.9 MB/s eta 0:00:00
Using cached namex-0.0.8-py3-none-any.whl (5.8 kB)
Downloading optree-0.14.1-cp311-cp311-win_amd64.whl (305 kB)
 305.8/305.8 kB 3.7 MB/s eta 0:00:00
Downloading rich-14.0.0-py3-none-any.whl (243 kB)
 243.2/243.2 kB 3.0 MB/s eta 0:00:00
Using cached markdown_it_py-3.0.0-py3-none-any.whl (87 kB)
Using cached MarkupSafe-3.0.2-cp311-cp311-win_amd64.whl (15 kB)
Downloading pygments-2.19.1-py3-none-any.whl (1.2 MB)
 1.2/1.2 MB 4.3 MB/s eta 0:00:00
Using cached mdurl-0.1.2-py3-none-any.whl (10.0 kB)
Installing collected packages: namex, libclang, flatbuffers, wrapt, wheel, urlli
flow-io-gcs-filestorage, tensorboard-data-server, six, pygments, protobuf, packag
e, markdown, idna, grpcio, gast, charset-normalizer, certifi, absl-py, werkzeug,
t-py, h5py, google-pasta, astunparse, tensorboard, rich, keras, tensorflow
Successfully installed MarkupSafe-3.0.2 absl-py-2.2.2 astunparse-1.6.3 certifi-2
ffers-25.2.10 gast-0.6.0 google-pasta-0.2.0 grpcio-1.71.0 h5py-3.13.0 idna-3.10
markdown-it-py-3.0.0 mdurl-0.1.2 ml-dtypes-0.5.1 namex-0.0.8 numpy-2.1.3 opt-ei
protobuf-5.29.4 pygments-2.19.1 requests-2.32.3 rich-14.0.0 six-1.17.0 tensorboa
tensorflow-2.19.0 tensorflow-io-gcs-filestorage-0.31.0 termcolor-3.0.1 typing-ext
1.3 wheel-0.45.1 wrapt-1.17.2

[notice] A new release of pip is available: 24.0 -> 25.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

What is a Tensor?

- Tensors generalize scalars, vectors, and matrices
- Multi-dimensional arrays that transform under coordinate changes
- Used to describe direction-dependent physical quantities
- Rank defines type:
- Rank 0: Scalars (e.g., temperature)
- Rank 1: Vectors (e.g., force)
- Rank 2: Matrices (e.g., stress, strain)



Source



Scalars, Vectors, and Tensors

- Scalars: Magnitude only (e.g., time, temperature) → Rank 0
- Vectors: Magnitude + direction (e.g., velocity, force) → Rank 1
- Tensors: Multi-dimensional arrays; extend vectors to higher ranks
- Rank = Number of indices needed to define the tensor
 - Rank 0: Scalar
 - Rank 1: Vector
 - Rank 2: Matrix
 - Rank 3+: Higher-dimensional tensors for complex interactions

Scalar



0D tensor

Vector



1D tensor

Matrix



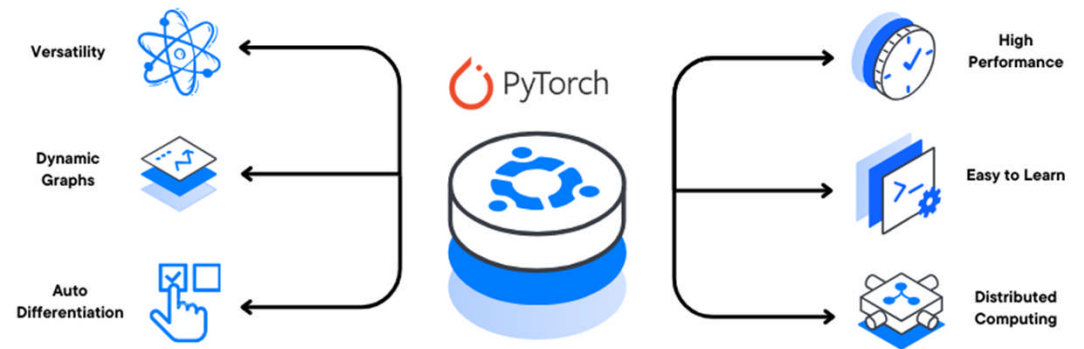
2D tensor

Source

<https://medium.com/data-science/what-is-a-tensor-in-deep-learning-6dedd95d6507>

Introduction to PyTorch

- PyTorch is an open-source machine learning library developed by Facebook's AI Research Lab (FAIR).
- It uses dynamic computation graphs, making it easier to build, modify, and debug models during runtime.
- PyTorch provides essential tools for everything from simple linear models to complex neural networks.



Source

Source: <https://docs.vultr.com/how-to-install-pytorch-on-ubuntu-22-04>



Why Use PyTorch?



Dynamic Computation Graphs

Build and modify neural networks as you go.
Great for debugging and experimentation.



Deep Python Integration

Feels natural to use for Python developers.



Strong Community

Growing ecosystem of tutorials, tools, and pretrained models.



GPU Acceleration

Seamless integration with CUDA for faster model training using GPUs.



Introduction to PyTorch

To install PyTorch, you can use either pip or conda. The command may vary slightly based on your system and whether you want GPU support.

For CPU only:

```
pip install torch torchvision
```

```
PowerShell 7.5.1  
Loading personal and system profiles took 1226ms.  
PS C:\Users\bhawa> pip install torch torchvision
```

Tensors: The Building Blocks: PyTorch

In PyTorch, the fundamental data structure is the Tensor, which is similar to NumPy arrays but with additional capabilities such as GPU acceleration.

```
1 import torch
2
3 # Creating a tensor from a list
4 a = torch.tensor([1, 2, 3])
5 print(a)
6 # Creating a 2D tensor
7 b = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
8 print(b)
9 # Creating tensors filled with zeros or ones
10 zeros = torch.zeros(2, 3)
11 ones = torch.ones(2, 3)
12 print(zeros)
13 print(ones)
14 # Random tensor
15 rand_tensor = torch.rand(2, 2)
16 print(rand_tensor)
```

[5] ✓ 0.0s Open 'rand_tensor' in Data Wrangler

Output

```
... tensor([1, 2, 3])
    tensor([[1., 2.],
            [3., 4.]])
    tensor([[0., 0., 0.],
            [0., 0., 0.]])
    tensor([[1., 1., 1.],
            [1., 1., 1.]])
    tensor([[0.4707, 0.3641],
            [0.2074, 0.6524]])
```



Basic Operations on tensors : PyTorch



```
1 x = torch.tensor([1.0, 2.0])
2 y = torch.tensor([3.0, 4.0])
3
4 print(x + y)      # Element-wise addition
5 print(x * y)      # Element-wise multiplication
6 print(torch.dot(x, y)) # Dot product
7
```

[6] ✓ 0.0s

```
... tensor([4., 6.])
    tensor([3., 8.])
    tensor(11.)
```



Lab Activity

Hands On

Lab 3

Lab 1.3 Customer Segmentation Using K-Means Clustering





Lab Activity

Hands On

Lab 4

Lab 1.4 Wind Power Generation Forecasting using Linear Regression and Random forest regressor





Lab Activity

Hands On

Lab 5

Lab 1.5 Crop Prediction using Decision Tree



Summary

- Unsupervised Learning: Learning from unlabeled data to uncover hidden patterns.
- Clustering: Grouping data points based on similarity without predefined labels.
- K-Means Clustering: A centroid-based method that iteratively partitions data into K clusters.
- Hierarchical Clustering: A tree-structured approach that builds nested groupings.
- TensorFlow: An open-source deep learning framework optimized for scalability and performance.
- PyTorch: A flexible deep learning library known for dynamic computation graphs and ease of debugging.



Source



References

- <https://www.datacamp.com/blog/introduction-to-unsupervised-learning>
- <https://www.analyticsvidhya.com/blog/2019/08/comprehensive-guide-k-means-clustering/>
- <https://www.geeksforgeeks.org/machine-learning/hierarchical-clustering/>
- <https://www.datacamp.com/tutorial/hierarchical-clustering>
- <https://www.geeksforgeeks.org/python/introduction-to-tensorflow/>
- <https://www.tensorflow.org/learn>
- https://docs.pytorch.org/tutorials/beginner/nlp/ptorch_tutorial.html
- <https://www.geeksforgeeks.org/deep-learning/pytorch-learn-with-examples/>





Quiz

1. What does the k in K-means clustering represent?

- a) The number of clusters to form
- b) The number of features in the dataset
- c) The number of iterations to run the algorithm
- d) The number of data points to sample



Answer: a

The number of clusters to form

Quiz

2. Which of the following techniques is used to reduce the dimensionality of data?

- a) K-means clustering
- b) t-SNE (t-distributed Stochastic Neighbor Embedding)
- c) Random Forest
- d) Logistic Regression

Answer: b

t-SNE (t-distributed Stochastic Neighbor Embedding)



Quiz

3. Which algorithm is commonly used for dimensionality reduction?

- a) K-means clustering
- b) Principal Component Analysis (PCA)
- c) Decision Trees
- d) Naive Bayes

Answer: b

Principal Component Analysis (PCA)



Quiz

4. In the context of hierarchical clustering, what does a dendrogram represent ?

- a) The error rate of the clustering process
- b) The visual representation of the data clusters at various levels
- c) The probability of each data point belonging to a cluster
- d) The total number of clusters formed

Answer: b

The visual representation of the data clusters at various levels





Quiz

5. What is the main difference between supervised and unsupervised learning ?

- a) Supervised learning uses labeled data, while unsupervised learning does not
- b) Unsupervised learning can only handle numerical data
- c) Supervised learning is faster than unsupervised learning
- d) Unsupervised learning requires more labeled data than supervised learning



Answer: a

Supervised learning uses labeled data, while unsupervised learning does not

Thank You